AsOf

A policy time machine and audit trail for AI assistants, built on IBM Db2

What did the policy say on that date, and what did our AI tell the customer?

IBM Db2 12.1.5 | **LangChain + LangGraph** | **Open source, Apache-2.0**

Jay Malhotra | solo entry | IBM Db2Hack 2026, TechX Summit | Round 1 proposal

github.com/JustJay7/asof

Companies own what their AI assistant says



Moffatt v. Air Canada

2024 BCCRT 149, decided 14 Feb 2024

The airline's website chatbot told a grieving customer he could claim a bereavement fare after travelling. The airline's own policy said he could not. The tribunal held the airline liable and rejected the argument that the chatbot was responsible for its own words.

CA$812

ordered in damages, interest and fees, over one chatbot answer



The company owns every statement

The tribunal rejected the defence that the correct policy was published on another page.



Policies change, answers drift

Refund windows, fees and eligibility rules change. Bots, cached FAQ answers and help articles keep repeating the old version.



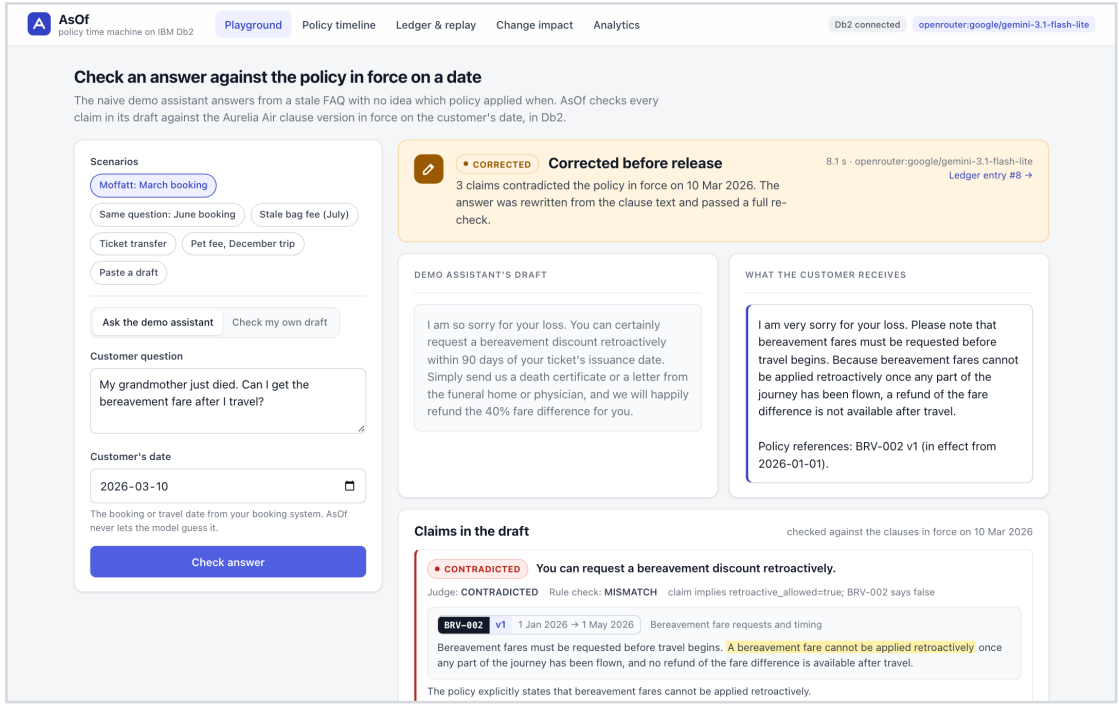
No evidence when it counts

In a dispute, teams cannot show which policy version an answer relied on, or what their systems knew at that moment.

AsOf: the policy in force on the customer's date

Given an AI-generated answer and a relevant date, AsOf finds the applicable policy version, flags claims that contradict it, stops unsupported answers from going out, and keeps the evidence needed to review that decision later.

- **As-of lookup.** Db2 returns the clause versions in force on the customer's date.
- **Claim check.** LLM judge, verbatim-quote rule, and deterministic rules on rule_json.
- **Decision.** PASS with citations, CORRECTED and re-checked, or ESCALATED. Fails closed.
- **Audit ledger.** Hash-chained Db2 record of every decision, replayable.
- **Change impact.** Past answers and help-centre content a policy change makes stale.



The playground on a March booking: the demo assistant's retroactive promise is corrected from clause BRV-002 v1 before it reaches the customer.

How one answer flows through AsOf

1

Answer in

Draft, question and the customer's date from the calling system. The model never picks the date.

2

Extract claims

Structured output: up to 8 atomic policy claims.

3

Retrieve as of date

One Db2 query: FOR BUSINESS_TIME AS OF plus VECTOR_DISTANCE.

4

Judge and check

One batched judge call. Quotes must be verbatim. rule_json overrides the model.

5

Decide

PASS, CORRECTED with a full re-check, or ESCALATED. Errors fail closed.

6

Record

APPEND_LEDGER writes a hash-chained row with citations and redacted text.

10 Mar 2026	*"You can claim the bereavement fare after you travel"*	**BRV-002 v1: request before travel**	**CORRECTED**
2 Jun 2026	*"You can claim the bereavement fare after you travel"*	**BRV-002 v2: retroactive within 90 days**	**PASS**

Same question, same words, different dates: the answer is right or wrong depending on which version was in force.

Architecture

Any AI assistant

Demo: a naive bot on LangChain with a DB2VS FAQ store

AsOf API (FastAPI)

Strict Pydantic validation, rate limit, security headers

Web UI

Playground, policy timeline, ledger and replay, change impact, analytics

Check pipeline

LangChain structured outputs (extract, batched judge, rewrite), deterministic rule checker, LangGraph agent with Db2 tools (in progress). Models: OpenRouter google/gemini-3.1-flash-lite by default, or local Ollama qwen2.5:7b. Embeddings: all-MiniLM-L6-v2 on CPU.

IBM Db2 12.1.5, schema ASOF

Users: asofapp (runtime), asofedit (policy changes), db2inst1 (setup only)

Policy

POLICY_DOC, POLICY_CLAUSE + history: bitemporal, VECTOR(384), RULE_JSON

Ledger

ANSWER_LEDGER, CLAIM_CITATION, CHAIN_HEAD: append-only, hash-chained

Content

PUBLISHED_ANSWER (help centre), FAQ vector store (DB2VS)

Routines and views

4 SQL PL procedures, SHA-256 functions, analytics views

Agentic AI: LangChain now, LangGraph agents next

BUILT

LangChain

- Structured-output extraction, one batched judge call, rewrite from clause text
- Custom BaseRetriever over the temporal and vector SQL
- Official langchain-db2 DB2VS store for the demo assistant
- Drop-in Runnable: assistant_chain | asof.as_runnable(as_of=date)

BY 9 NOV

LangGraph

- The check as a graph whose tools query Db2: as-of retriever, rule checker, ledger, change impact
- An investigate step for uncovered claims, capped at 3 tool calls
- Change-impact agent: drafts corrected help-centre answers, re-checks each, and queues them for approval
- Provider fallback: OpenRouter, then local Ollama, recorded in the ledger

Safety properties no agent step can bypass

Fixed graph edges | **Rules override the model** | **Verbatim quotes only** | **Full re-check before release** | **Any error means ESCALATED**

Innovation: adding time to answer checking

AsOf does not claim to have invented policy checking. What is new is time: what was true on date X, what our database knew at time T, and which past answers a change makes stale.

Closest existing tool	What it does	What AsOf adds
Amazon Bedrock Automated Reasoning checks	Validates answers against policies, suggests corrections, logs results	Effective-dated versions, replay of what the database knew, change impact, inside your own Db2, open source
NVIDIA NeMo Guardrails fact-checking rails	Scores an answer against retrieved chunks	Policy versions and dates, plus deterministic checks on structured rule parameters
IBM watsonx.governance	RAG quality metrics such as faithfulness	Per-answer evidence: which clause version, quoted, in a tamper-evident record



Effective-dated

Every clause version carries the dates it applied.



Bitemporal replay

FOR SYSTEM_TIME AS OF shows what the database held at answer time.



Change impact

A policy change lists the past answers and articles it makes stale.

Db2 integration: the database does the hard work

Bitemporal tables

BUSINESS_TIME and SYSTEM_TIME periods, a history table, a WITHOUT OVERLAPS key.

FOR PORTION OF

A policy change splits the version row; the old row moves to history.

Native VECTOR

VECTOR(384, FLOAT32) ranked with VECTOR_DISTANCE inside the temporal query.

SQL PL procedures

APPEND_LEDGER, VERIFY_CHAIN, APPLY_POLICY_CHANGE, CHANGE_IMPACT.

SHA-256 in SQL

HASH(..., 2) chains every ledger row inside the procedure.

JSON rule parameters

JSON_EXISTS checks on every rule; JSON_TABLE in views and the citation shredder.

Triggers

Block UPDATE and DELETE on the ledger, even for the owner.

Least privilege

Runtime user can only SELECT and EXECUTE; PUBLIC privileges revoked.

Connectors: ibm_db / ibm_db_dbi with bound parameters, the official langchain-db2 DB2VS vector store, and a custom LangChain retriever over the temporal and vector SQL.
Proof: python -m asof.db2check runs 26 of 26 Db2 feature checks on a fresh database.

One query answers: what applied on that date?

```
SELECT c.CLAUSE_ID, c.VERSION_LABEL, c.CLAUSE_TEXT,
       c.RULE_JSON,
       VECTOR_DISTANCE(c.EMBEDDING,
         VECTOR(?, 384, FLOAT32), COSINE) AS DIST
FROM   POLICY_CLAUSE
       FOR BUSINESS_TIME AS OF ? AS c
WHERE  c.DOC_ID = ?
ORDER  BY DIST
FETCH  FIRST ? ROWS ONLY
```

Temporal filter and vector ranking in one statement. Every value is a bound parameter.

Example: raise the pet fee to $125 from 1 Dec 2026

Nov answer: still valid | **Dec answer: stale** | **Help article: review**

Replay

Add FOR SYSTEM_TIME AS OF the answer's timestamp and Db2 returns exactly what it held when the bot answered, even if the policy was edited later.

Change impact

APPLY_POLICY_CHANGE updates FOR PORTION OF BUSINESS_TIME. CHANGE_IMPACT then lists each past answer as STALE or HISTORICALLY VALID, and each help article that needs review.

Feasibility: built, tested and measured

0 of 26

wrong drafts released, hosted model, frozen test split

92.5%

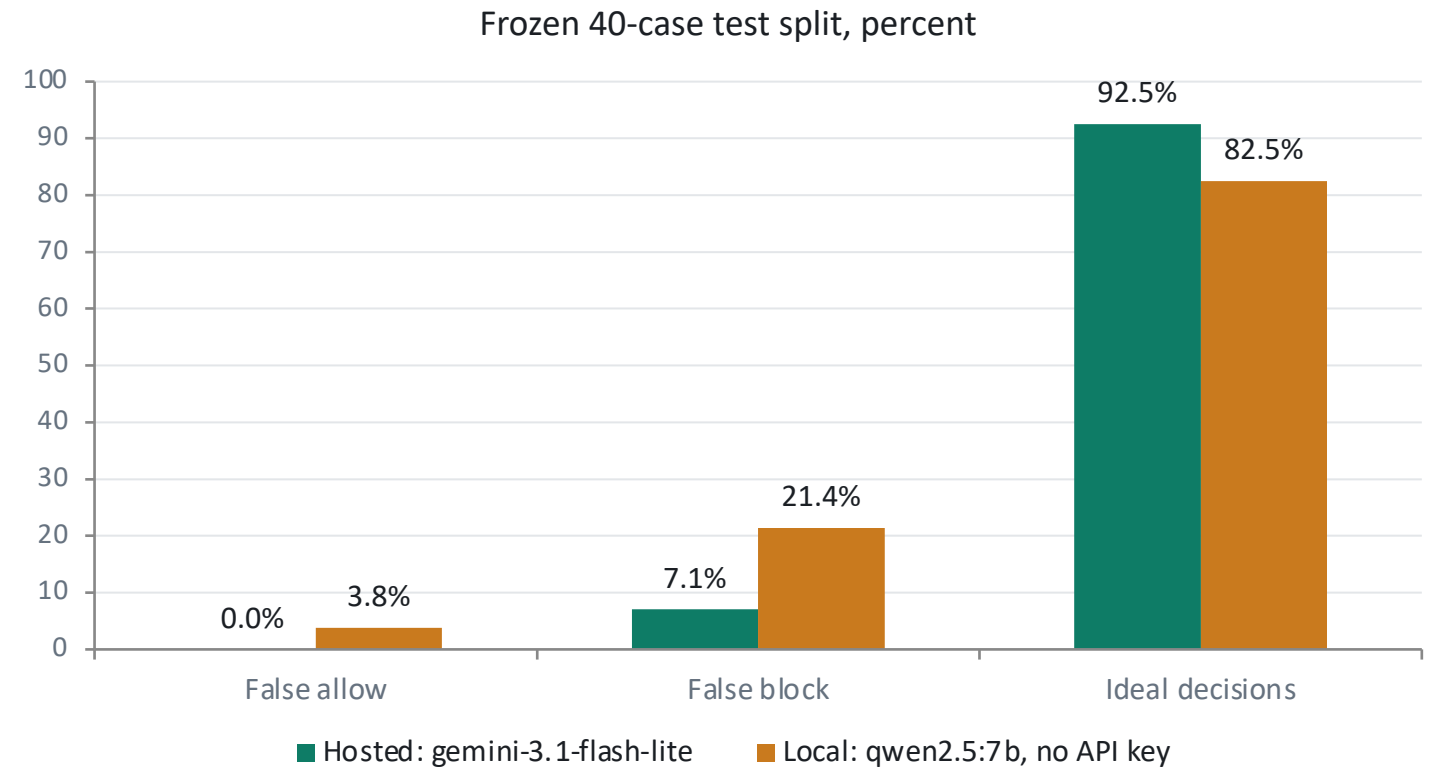
decisions exactly as labelled

100%

key claims checked against the right policy version

215

automated tests; 26 of 26 Db2 feature checks pass



The 40 test cases were committed and hash-pinned before their first run. Hosted runs take 5.9 s per answer; the local model takes 37.9 s on a MacBook Air. A fresh clone following only the README passes every test.

Security and trust by design

Fail closed

Any error, timeout or invalid model output becomes ESCALATED. No ledger record, no release.

Least privilege

The runtime user can only SELECT and EXECUTE. Ledger writes go only through APPEND_LEDGER.

Parameterised SQL

Every value is a bound parameter. A build-time test against string-built SQL lands by 9 Nov.

Prompt-injection handling

Customer and clause text are delimited data. Strict output schemas. Rule checks override the model.

Personal data redaction

Emails, phone numbers, card and booking numbers are redacted before anything is stored.

Tamper evidence

Triggers block UPDATE and DELETE. A SHA-256 hash chain lets VERIFY_CHAIN find the first altered row.

Expected impact

For CX platform, compliance and legal-ops teams at airlines, banks, insurers and telcos: organisations that already run Db2.



Before release

Wrong policy promises are corrected from the clause text, or held for a specialist, with citations.



In a dispute

Replay shows the exact policy version and what the database knew at that moment, in seconds.



After a policy change

Every past answer and help article the change makes stale is listed, with the date it went stale.

How it gets adopted: one REST endpoint, or one LangChain Runnable, in front of any assistant. Runs on Db2 Community Edition. Open source under Apache-2.0. A second domain, a bank fee schedule, is going through the same engine next.

The demo airline, Aurelia Air, and its policies are fictional, modelled on the policy at issue in Moffatt.

Status and next steps

Built and measured

- Check pipeline with fail-closed decisions
- Db2 schema, procedures, triggers and views
- Hash-chained ledger with replay
- Change impact on answers and help-centre content
- Five-screen web UI
- Frozen eval on two providers; 215 tests

Round 2 build, by 9 Nov

- LangGraph agents with Db2 tools
- Native vector index, benchmarked against exact search
- In-database embeddings with TO_EMBEDDING
- Ask the ledger, with row and column access control
- A bank fee schedule as a second domain
- CI, dbt, Airflow and Langflow integrations

github.com/JustJay7/asof

Jay Malhotra | IBM Db2Hack 2026